

Estructuras de datos geoespaciales

Anamaria Rodríguez Ramirez

2026-04-01

Contenido

1	Ejercicio 1: Topología del Río Cauca (Líneas y Puntos)	3
1.1	Lenguaje Python	3
1.2	Lenguaje R	3
1.3	Lenguaje Julia	4
1.4	Ejercicio 2	6
1.5	Lenguaje Python	6
1.6	Lenguaje R	7
1.7	Lenguaje Julia	7
2	Entregables y Criterios de Evaluación	10
3	Referencias	11

```
# #| include: false  
source("/home/rstudio/work/ejercicios/datos_geoespaciales/j_eval_j_plot-previa4.r")
```

Los siguientes ejercicios fueron desarrollados en base al capítulo Estructura de Datos Geoespaciales del curso Programación en SIG: R, Python y Julia, creado por el docente Rodríguez-Avellaneda, 2026. El objetivo es que puedas practicar la creación de objetos geométricos básicos (Puntos, Líneas, Polígonos) y la conversión de tablas planas a tablas espaciales con un sistema de referencia.

1 Ejercicio 1: Topología del Río Cauca (Líneas y Puntos)

Contexto: Estás analizando estaciones de monitoreo de calidad del agua sobre un tramo del Río Cauca. Tienes las coordenadas de las estaciones y necesitas construir los objetos geométricos individuales para luego incluirlos en tu reporte base.

Instrucciones de código:

1. Utilizando las herramientas de creación de geometrías nativas de tu lenguaje, crea un Punto
2. Crea un segundo Punto para la Estación Mediacanoa (Lon: -76.38, Lat: 3.88). Guárdalo en la v
3. Imagina que el río conecta ambos puntos en línea recta. Crea una geometría tipo Línea (LineS
4. Imprime el tipo de geometría (o la representación en texto/WKT) de tu variable tramo_cauca.

1.1 Lenguaje Python

```
import geopandas as gpd
from shapely.geometry import Point, LineString

pt_juanchito = Point(-76.45, 3.45)
pt_mediacaño = Point(-76.38, 3.88)

tramo_cauca = LineString([pt_juanchito, pt_mediacaño])

print(f"Tipo de geometría: {tramo_cauca.geom_type}")
```

Tipo de geometría: LineString

```
print(f"Representación WKT: {tramo_cauca}")
```

Representación WKT: LINESTRING (-76.45 3.45, -76.38 3.88)

1.2 Lenguaje R

```
library(sf)
```

Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE

```
library(sfheaders)
```

```
pt_juanchito <- st_point(c(-76.45, 3.45))
```

```
pt_mediacaño <- st_point(c(-76.38, 3.88))
```

```
tramo_cauca <- st_linestring(rbind(st_coordinates(pt_juanchito), st_coordinates(pt_mediacaño)))
```

```
print(paste("Tipo de geometría:", st_geometry_type(tramo_cauca)))
```

```
[1] "Tipo de geometría: LINESTRING"
```

```
print(paste("Representación WKT:", st_as_text(tramo_cauca)))
```

```
[1] "Representación WKT: LINESTRING (-76.45 3.45, -76.38 3.88)"
```

1.3 Lenguaje Julia

```
j_eval(r"-(
```

```
using LibGEOS
```

```
pt_juanchito = LibGEOS.readgeom("POINT(-76.45 3.45)")
```

```
pt_mediacaño = LibGEOS.readgeom("POINT(-76.38 3.88)")
```

```
tramo_cauca = LibGEOS.readgeom("LINESTRING(-76.45 3.45, -76.38 3.88)")
```

```
using Printf
```

```
println("Tipo de geometría: $(LibGEOS.geomTypeId(tramo_cauca))")
```

```
println("Representación WKT: $tramo_cauca")
```

```
)-")
```

Starting Julia ...

```

<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
<span style='color:#28a745; font-weight:bold;'>julia> </span><span style='color:#008b8b;'>using Plots, GeoPkg
</pre>
<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
</pre>
<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
<span style='color:#28a745; font-weight:bold;'>julia> </span><span style='color:#008b8b;'>plot(P, "lon", "lat", "name")
</pre>
<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
POINT (-76.45 3.45)

</pre>
<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
<span style='color:#28a745; font-weight:bold;'>julia> </span><span style='color:#008b8b;'>plot(P, "lon", "lat", "name")
</pre>
<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
POINT (-76.38 3.88)

</pre>
<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
<span style='color:#28a745; font-weight:bold;'>julia> </span><span style='color:#008b8b;'>plot(P, "lon", "lat", "name")
</pre>
<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
LINESTRING (-76.45 3.45, -76.38 3.88)

</pre>
<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
<span style='color:#28a745; font-weight:bold;'>julia> </span><span style='color:#008b8b;'>using Plots, GeoPkg
</pre>
<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
Tipo de geometría: 1

</pre>
<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
<span style='color:#28a745; font-weight:bold;'>julia> </span><span style='color:#008b8b;'>plot(P, "lon", "lat", "name")
</pre>
<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 0 10px; margin-bottom:10px 0 0 0;'>
Representación WKT: LINESTRING (-76.45 3.45, -76.38 3.88)

</pre>

```

En Julia, se imprime en el tipo de geometría como un número entero, donde 0 corresponde a `Point`, 1 a `LineString`, y así sucesivamente. En este caso, el resultado será 1, indicando que la geometría es un `LineString`, debido a que esta librería está escrito en C++, por lo empieza su conteo desde 0 y no desde 1 como en R o Julia.

1.4 Ejercicio 2

Contexto: El equipo de Gestión del Riesgo te ha entregado un archivo tabular tradicional (CSV) con los barrios que reportan riesgo de inundación y sus coordenadas. Tu misión es convertir esta tabla “plana” en un objeto espacial oficial con un sistema de referencia.

Instrucciones de código:

1. Crea un `DataFrame` clásico con la siguiente información:
 - Columna Barrio: "La Virginia", "Puerto Mallarino"
 - Columna Riesgo: "Alto", "Medio"
 - Columna Lon: -75.88, -76.21
 - Columna Lat: 4.90, 3.95
2. Utiliza la función apropiada de tu librería espacial (`GeoPandas`, `sf` o `GeoTables`) para convertir.
3. Al realizar la conversión, asigna explícitamente el sistema de coordenadas geográficas estándar.
4. Imprime en consola tu nueva tabla espacial.

1.5 Lenguaje Python

```
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point

df = pd.DataFrame({
    "Barrio": ["La Virginia", "Puerto Mallarino"],
    "Riesgo": ["Alto", "Medio"],
    "Lon": [-75.88, -76.21],
    "Lat": [4.90, 3.95]
})

geoms = [Point(xy) for xy in zip(df["Lon"], df["Lat"])]

gdf = gpd.GeoDataFrame(df, geometry=geoms, crs="EPSG:4326")

print(gdf)
```

	Barrio	Riesgo	Lon	Lat	geometry
0	La Virginia	Alto	-75.88	4.90	POINT (-75.88 4.9)
1	Puerto Mallarino	Medio	-76.21	3.95	POINT (-76.21 3.95)

1.6 Lenguaje R

```
library(sf)

df <- data.frame(
  Barrio = c("La Virginia", "Puerto Mallarino"),
  Riesgo = c("Alto", "Medio"),
  Lon = c(-75.88, -76.21),
  Lat = c(4.90, 3.95)
)

gdf <- st_as_sf(df, coords = c("Lon", "Lat"), crs = 4326)
print(gdf)
```

Simple feature collection with 2 features and 2 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: -76.21 ymin: 3.95 xmax: -75.88 ymax: 4.9

Geodetic CRS: WGS 84

	Barrio	Riesgo	geometry
1	La Virginia	Alto	POINT (-75.88 4.9)
2	Puerto Mallarino	Medio	POINT (-76.21 3.95)

1.7 Lenguaje Julia

```
j_eval(r"-(
using DataFrames, GeoTables, Meshes

df = DataFrame(
  Barrio = ["La Virginia", "Puerto Mallarino"],
  Riesgo = ["Alto", "Medio"],
  Lon = [-75.88, -76.21],
  Lat = [4.90, 3.95] )

puntos = [Meshes.Point(df.Lon[i], df.Lat[i]) for i in 1:nrow(df)]

gt = georef(df, puntos)
println(gt)

)-")
```

```
<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px; margin-bottom:10px; font-family:monospace; font-size:1em; color:#333333; white-space:pre-wrap; overflow:auto; min-height:100px; width:100%;>
```

julia> us
</pre>

<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 10px 10px;'>

</pre>

<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 10px 10px;'>

julia> df

 Barrio = ["La Virginia", "Puerto Mallarino"],

 Riesgo = ["Alto", "Medio"],

 Lon = [-75.88, -76.21],

 Lat = [4.90, 3.95])

</pre>

<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 10px 10px;'>

2×4 DataFrame

| Row | Barrio
String | Riesgo
String | Lon
Float64 | Lat
Float64 |
|-----|------------------|------------------|----------------|----------------|
| 1 | La Virginia | Alto | -75.88 | 4.9 |
| 2 | Puerto Mallarino | Medio | -76.21 | 3.95 |

</pre>

<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 10px 10px;'>

julia> pu

</pre>

<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 10px 10px;'>

2-element Vector{Meshes.Point{ {2}, CoordRefSystems.Cartesian2D{CoordRefSystems.NoDatum, Unitful{<div data-bbox="119 535 387 551" data-label="Text">

Point(x: -75.88 m, y: 4.9 m)

Point(x: -76.21 m, y: 3.95 m)

</pre>

<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 10px 10px;'>

julia> gt

</pre>

<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 10px 10px;'>

2×5 GeoTable over 2 PointSet

| Barrio
Categorical
[NoUnits] | Riesgo
Categorical
[NoUnits] | Lon
Continuous
[NoUnits] | Lat
Continuous
[NoUnits] | geometry
Point
Cartesian{NoD |
|------------------------------------|------------------------------------|--------------------------------|--------------------------------|------------------------------------|
| La Virginia | Alto | -75.88 | 4.9 | (x: -75.88 m, y: |
| Puerto Mallarino | Medio | -76.21 | 3.95 | (x: -76.21 m, y: |

1 column omitted

</pre>

<pre class='julia-code' style='display:block; background:#f4f6f8; border:1px solid #c0c8d0; border-radius:5px; padding:10px 10px 10px 10px;'>

julia> pr

```
</pre>
```

```
<pre class='julia-output' style='display:block; background:#ffffff; border:none; padding:10px 10px 10px 10px'>
2×5 GeoTable over 2 PointSet
```

```
</pre>
```

2 Entregables y Criterios de Evaluación

****Sobre la Topología (Ejercicio 1):**** Basado en la lectura y tu experiencia práctica, ¿por qué

Es importante respetar el orden (Longitud, Latitud) porque las librerías y el tipo de formato utilizado, operan sobre un plano cartesiano, es decir coordenadas (X,Y). Si se interpretara cómo suele mencionarse las coordenadas, (Latitud, Longitud) otra manera, el lenguaje interpretaría la Latitud como Longitud y viceversa por lo que el punto estaría ubicado en un lugar completamente distinto al que se esperaría.

Además, X, es el eje horizontal, que corresponde a la longitud que va de este a oeste, mientras que Y es el eje vertical, que corresponde a la latitud que va de norte a sur. Si se invierte el orden, se estaría asignando la latitud al eje X y la longitud al eje Y, lo que resultaría en una ubicación geográfica incorrecta.

****Sobre la Proyección:**** El texto de la lección hace una advertencia sobre calcular áreas usando

Porque no es una medida que permita determinar distancias, las coordenadas de referencia en formato EPSG:4326, al encontrarse en grados no permite tener un dato de distancia entre dos puntos, sino que al ser una medida que corresponde a una unidad que calcula el ángulo de un vértice entre el centro y sus lados, el resultado sería un ángulo, no una distancia. Por otra parte, el formato EPSG:9377 (Magna-Sirgas/Origen Nacional), es un sistema de coordenadas proyectado que utiliza metros como unidad de medida, lo que permite calcular áreas y distancias de manera precisa y útil para la ingeniería y el planeamiento territorial en Colombia.

3 Referencias

Rodríguez-Avellaneda, A. H. (2026). Programación en SIG: R, Python y Julia [Material de curso]. Maestría en Geomática, Universidad Nacional de Colombia. Recuperado de <https://github.com/alexyshr/ProgramacionSIG2026>